

LangChain: The Complete Guide

From Zero to Production & Enterprise — Beginner to Intermediate

Goal: By the end of this guide, you will understand LangChain deeply enough to build real-world AI applications and confidently answer any interview question — including production and enterprise scenarios.

Table of Contents

1. [What is LangChain? \(The Big Picture\)](#)
 2. [LangChain Architecture & Core Concepts](#)
 3. [Setup & Installation](#)
 4. [LLMs & Chat Models](#)
 5. [Prompt Templates](#)
 6. [Chains — The Heart of LangChain](#)
 7. [Output Parsers](#)
 8. [Document Loaders](#)
 9. [Text Splitters](#)
 10. [Embeddings & Vector Stores](#)
 11. [Retrievers & RAG \(Retrieval-Augmented Generation\)](#)
 12. [Memory — Giving AI a Brain](#)
 13. [Agents & Tools](#)
 14. [LangChain Expression Language \(LCEL\)](#)
 15. [LangSmith — Observability & Debugging](#)
 16. [LangServe — Deploying LangChain Apps as APIs](#)
 17. [LangGraph — Stateful Multi-Agent Workflows](#)
 18. [Production & Enterprise Patterns](#)
 19. [Common Interview Q&A](#)
 20. [Best Resources & Learning Path](#)
-

1. What is LangChain?

The Simple Explanation

Imagine you want to build an AI app that:

- Reads your company's PDFs
- Answers questions from them
- Remembers what you talked about earlier
- Searches the web when needed
- Makes decisions on what to do next

Doing all of this from scratch with just an LLM API is painful. **LangChain is a framework that gives you ready-made building blocks to compose all of this together.**

The Official Definition

LangChain is an open-source framework for building applications powered by Large Language Models (LLMs). It provides:

- **Abstractions** over LLM providers (OpenAI, Anthropic, Google, Mistral, etc.)
- **Components** for common tasks (prompting, retrieval, memory, agents)
- **Integrations** with 100+ tools, databases, and APIs
- **Composability** — chain components together like Lego blocks

Why LangChain Exists (The Problem It Solves)

Without LangChain	With LangChain
Write raw API calls to each LLM	Single interface for all LLMs
Manually parse LLM output	Structured output parsers
Build your own vector DB integration	50+ built-in vector store integrations
Roll your own memory logic	Plug-in memory modules
Complex agent loop from scratch	Agents with tools in 10 lines

LangChain Ecosystem (2024-2025)

langchain-core	→ Core abstractions (base classes, LCEL)
----------------	--

langchain → Main framework (chains, agents, retrievers)
langchain-community → 3rd-party integrations (vector stores, tools)
langchain-openai → OpenAI-specific package
langchain-anthropic → Anthropic-specific package
langsmith → Observability, tracing, evaluation
langgraph → Stateful multi-agent graphs
langserve → Deploy chains as REST APIs

2. Architecture & Core Concepts

The 6 Core Building Blocks

INPUT

↓

[Prompt Template] → Formats your input into a proper prompt

↓

[LLM / Chat Model] → Sends to AI and gets response

↓

[Output Parser] → Structures the response (JSON, list, etc.)

↓

[Memory] → Optionally stores/retrieves conversation history

↓

[Retriever] → Optionally fetches relevant documents

↓

[Agent/Tool] → Optionally decides what action to take

OUTPUT

The Runnable Protocol (LCEL)

Everything in modern LangChain implements the `Runnable` interface, meaning every component has:

- `.invoke(input)` — run once
- `.stream(input)` — stream output token by token
- `.batch(inputs)` — run many inputs in parallel
- `.ainvoke(input)` — async version

This lets you chain them with `|` (pipe operator):

```
python
```

```
chain = prompt | llm | output_parser
result = chain.invoke({"question": "What is LangChain?"})
```

3. Setup & Installation

Installation

```
bash

# Core packages
pip install langchain langchain-core langchain-community

# LLM providers (install what you use)
pip install langchain-openai      # OpenAI & Azure OpenAI
pip install langchain-anthropic   # Claude models
pip install langchain-google-genai # Gemini models
pip install langchain-ollama      # Local models via Ollama

# Vector stores
pip install langchain-chroma      # ChromaDB
pip install langchain-pinecone    # Pinecone
pip install faiss-cpu             # FAISS (local, fast)

# Utilities
pip install python-dotenv         # Load .env files
pip install langsmith            # Observability
```

Environment Setup

```
python

# .env file
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=sk-ant-...
LANGCHAIN_API_KEY=ls__... # LangSmith
LANGCHAIN_TRACING_V2=true # Enable tracing
LANGCHAIN_PROJECT=my-project # LangSmith project name
```

```
python
```

```
# Python: load env vars
from dotenv import load_dotenv
load_dotenv()

import os
openai_key = os.getenv("OPENAI_API_KEY")
```

Verify Installation

```
python

from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o-mini")
response = llm.invoke("Say hello in 5 words")
print(response.content)
# Output: Hello there, how are you?
```

4. LLMs & Chat Models

Understanding the Difference

LLM (Text In → Text Out)

`OpenAI("text-davinci-003")`

Takes a string

Older style

`llm.invoke("Hello")` → `"Hi there"`

Chat Model (Messages In → Message Out)

`ChatOpenAI("gpt-4o")`

Takes a list of messages

Modern, recommended

`chat.invoke([HumanMessage("Hello")])` → `AIMessage("Hi there")`

Message Types

```
python
```

```

from langchain_core.messages import (
    SystemMessage, # Sets AI's persona/instructions
    HumanMessage, # User's message
    AIMessage, # AI's previous response
    FunctionMessage, # Tool/function result
)

messages = [
    SystemMessage(content="You are a helpful Python tutor."),
    HumanMessage(content="What is a list comprehension?"),
]

```

Using Different LLM Providers

```

python

# OpenAI
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model="gpt-4o", temperature=0.7, max_tokens=500)

# Anthropic Claude
from langchain_anthropic import ChatAnthropic
llm = ChatAnthropic(model="claude-3-5-sonnet-20241022")

# Google Gemini
from langchain_google_genai import ChatGoogleGenerativeAI
llm = ChatGoogleGenerativeAI(model="gemini-1.5-pro")

# Local with Ollama (free, runs on your machine)
from langchain_ollama import ChatOllama
llm = ChatOllama(model="llama3.2") # After: ollama pull llama3.2

```

Key Parameters

```
python
```

```
ChatOpenAI(  
    model="gpt-4o",  
    temperature=0,    # 0=deterministic, 1=creative, 2=chaotic  
    max_tokens=1000,  # Max output tokens  
    timeout=30,       # API timeout in seconds  
    max_retries=3,     # Auto-retry on failure  
    streaming=True,    # Enable token streaming  
)
```

Invoking Models

```
python  
  
# Simple invoke  
response = llm.invoke("What is 2+2?")  
print(response.content)    # "4"  
print(response.usage_metadata) # token counts  
  
# Streaming (shows tokens one by one)  
for chunk in llm.stream("Write a haiku about Python"):  
    print(chunk.content, end="", flush=True)  
  
# Async (for web servers)  
import asyncio  
response = asyncio.run(llm.ainvoke("Hello"))  
  
# Batch (parallel requests)  
responses = llm.batch(["Question 1", "Question 2", "Question 3"])
```

5. Prompt Templates

Why Templates?

Instead of building strings manually, templates let you:

- Reuse prompts with different inputs
- Maintain clean, readable code
- Use chat-style formatting easily

PromptTemplate (Simple String)

```
python

from langchain_core.prompts import PromptTemplate

# Method 1: from_template (recommended)
prompt = PromptTemplate.from_template(
    "You are an expert in {domain}. Answer this question: {question}"
)

# Format it
formatted = prompt.format(domain="Python", question="What is a decorator?")
print(formatted)
# "You are an expert in Python. Answer this question: What is a decorator?"

# Use with LLM
chain = prompt | llm
result = chain.invoke({"domain": "Python", "question": "What is a decorator?"})
```

ChatPromptTemplate (Messages)

```
python

from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful {role}. Always respond in {language}."),
    ("human", "{user_input}"),
])

# Invoke
chain = prompt | llm
result = chain.invoke({
    "role": "data scientist",
    "language": "English",
    "user_input": "Explain overfitting"
})
```

MessagesPlaceholder (For Memory/History)

```
python
```



```

from langchain_core.prompts import MessagesPlaceholder

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant."),
    MessagesPlaceholder(variable_name="chat_history"), # ← Insert history here
    ("human", "{input}"),
])

```

FewShotPromptTemplate (Give Examples)

```

python

from langchain_core.prompts import FewShotChatMessagePromptTemplate

examples = [
    {"input": "happy", "output": "sad"},
    {"input": "tall", "output": "short"},
    {"input": "hot", "output": "cold"},
]

example_prompt = ChatPromptTemplate.from_messages([
    ("human", "{input}"),
    ("ai", "{output}"),
])

few_shot_prompt = FewShotChatMessagePromptTemplate(
    examples=examples,
    example_prompt=example_prompt,
)

final_prompt = ChatPromptTemplate.from_messages([
    ("system", "Give antonyms for words."),
    few_shot_prompt,
    ("human", "{input}"),
])

chain = final_prompt | llm
result = chain.invoke({"input": "energetic"})
# Output: "lethargic" (learned the pattern from examples)

```

6. Chains

What is a Chain?

A chain is a sequence of operations. Think of it as a pipeline: input → step 1 → step 2 → ... → output.

Old vs. New Style

Old Style (LegacyChain): LLMChain, ConversationalChain (still works, but deprecated)

New Style (LCEL): prompt | llm | parser (recommended from LangChain v0.1+)

Basic LCEL Chain

```
python

from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

llm = ChatOpenAI(model="gpt-4o-mini")

prompt = ChatPromptTemplate.from_template(
    "Summarize this text in one sentence: {text}"
)

parser = StrOutputParser()

# The Chain (pipe operator connects them)
chain = prompt | llm | parser

# Run it
result = chain.invoke({"text": "LangChain is a framework for building LLM apps..."})
print(result) # "LangChain is a framework that simplifies building LLM-powered applications."
```

Sequential Chain (Multi-step)

```
python
```

```

# Step 1: Generate a blog title
title_prompt = ChatPromptTemplate.from_template(
    "Write a catchy blog title about: {topic}"
)

# Step 2: Write the blog post for that title
post_prompt = ChatPromptTemplate.from_template(
    "Write a 3-paragraph blog post with this title: {title}"
)

# Chain them: topic → title → blog post
chain = (
    title_prompt
    | llm
    | StrOutputParser()
    | (lambda title: {"title": title}) # reshape output
    | post_prompt
    | llm
    | StrOutputParser()
)

result = chain.invoke({"topic": "AI in healthcare"})

```

RunnableParallel (Run Multiple Things at Once)

```

python

from langchain_core.runnables import RunnableParallel

parallel_chain = RunnableParallel({
    "summary": prompt_summary | llm | StrOutputParser(),
    "key_points": prompt_bullets | llm | StrOutputParser(),
    "sentiment": prompt_sentiment | llm | StrOutputParser(),
})

# All three run in parallel
result = parallel_chain.invoke({"text": "Some long article..."})
# result = {"summary": "...", "key_points": "...", "sentiment": "..."}

```

RunnablePassthrough (Pass Input Through)

```

python

```

```
from langchain_core.runnables import RunnablePassthrough

chain = RunnableParallel({
    "original_question": RunnablePassthrough(), # Keep original input
    "answer": prompt | llm | StrOutputParser(),
})
```

RunnableLambda (Custom Functions in Chain)

```
python

from langchain_core.runnables import RunnableLambda

def clean_text(text):
    return text.strip().lower()

chain = RunnableLambda(clean_text) | prompt | llm | StrOutputParser()
```

Error Handling: .with_fallbacks()

```
python

# If GPT-4 fails (quota, timeout), fall back to GPT-3.5
robust_llm = ChatOpenAI(model="gpt-4o").with_fallbacks([
    ChatOpenAI(model="gpt-4o-mini")
])
```

Retry Logic: .with_retry()

```
python

llm_with_retry = llm.with_retry(
    stop_after_attempt=3,
    wait_exponential_jitter=True
)
```

7. Output Parsers

Why Parsers?

LLMs return text. But you usually want structured data — JSON, lists, objects. Parsers handle the conversion.

StrOutputParser (Simple Text)

```
python

from langchain_core.output_parsers import StrOutputParser
parser = StrOutputParser()
# Returns: plain string
```

JsonOutputParser (JSON Object)

```
python

from langchain_core.output_parsers import JsonOutputParser
from langchain_core.prompts import PromptTemplate

parser = JsonOutputParser()

prompt = PromptTemplate(
    template="Extract name, age, city from: {text}\n{format_instructions}",
    input_variables=["text"],
    partial_variables={"format_instructions": parser.get_format_instructions()}
)

chain = prompt | llm | parser
result = chain.invoke({"text": "John Smith is 30 years old from New York"})
# result = {"name": "John Smith", "age": 30, "city": "New York"}
```

PydanticOutputParser (Typed, Validated)

```
python
```

```

from langchain_core.output_parsers import PydanticOutputParser
from pydantic import BaseModel, Field
from typing import List

class Person(BaseModel):
    name: str = Field(description="Person's full name")
    age: int = Field(description="Person's age in years")
    skills: List[str] = Field(description="List of skills")

parser = PydanticOutputParser(pydantic_object=Person)

prompt = PromptTemplate(
    template="Extract person info from: {text}\n{format_instructions}",
    input_variables=["text"],
    partial_variables={"format_instructions": parser.get_format_instructions()}
)

chain = prompt | llm | parser
result = chain.invoke({"text": "Alice, 28, knows Python, SQL, and ML"})
# result = Person(name="Alice", age=28, skills=["Python", "SQL", "ML"])
print(result.name) # "Alice"
print(result.skills) # ["Python", "SQL", "ML"]

```

CommaSeparatedListOutputParser

```

python

from langchain.output_parsers import CommaSeparatedListOutputParser

parser = CommaSeparatedListOutputParser()
prompt = PromptTemplate(
    template="List 5 {item}s.\n{format_instructions}",
    input_variables=["item"],
    partial_variables={"format_instructions": parser.get_format_instructions()}
)
chain = prompt | llm | parser
result = chain.invoke({"item": "Python libraries"})
# result = ["numpy", "pandas", "scikit-learn", "matplotlib", "tensorflow"]

```

With Structured Output (Modern Approach)

```

python

```

```
# Direct structured output — LangChain handles everything
from pydantic import BaseModel

class Joke(BaseModel):
    setup: str
    punchline: str

structured_llm = llm.with_structured_output(Joke)
result = structured_llm.invoke("Tell me a programming joke")
# result = Joke(setup="Why do programmers prefer dark mode?", punchline="Because light attracts bugs!")
print(result.punchline)
```

8. Document Loaders

What are Document Loaders?

Loaders convert different file types and sources into LangChain `Document` objects.

A `Document` has two fields:

- `page_content` — the text
- `metadata` — info about the source (filename, page number, URL, etc.)

Common Loaders

```
python
```

Text file

```
from langchain_community.document_loaders import TextLoader
```

```
loader = TextLoader("data/story.txt")
```

```
docs = loader.load()
```

PDF

```
from langchain_community.document_loaders import PyPDFLoader
```

```
loader = PyPDFLoader("report.pdf")
```

```
docs = loader.load() # One Document per page
```

```
# pip install pypdf
```

CSV

```
from langchain_community.document_loaders import CSVLoader
```

```
loader = CSVLoader("data.csv", content_columns=["description"])
```

```
docs = loader.load() # One Document per row
```

Web page

```
from langchain_community.document_loaders import WebBaseLoader
```

```
loader = WebBaseLoader("https://docs.langchain.com")
```

```
docs = loader.load()
```

```
# pip install bs4
```

Directory (all files)

```
from langchain_community.document_loaders import DirectoryLoader
```

```
loader = DirectoryLoader("./documents/", glob="**/*.pdf", loader_cls=PyPDFLoader)
```

```
docs = loader.load()
```

Word Document (.docx)

```
from langchain_community.document_loaders import Docx2txtLoader
```

```
loader = Docx2txtLoader("report.docx")
```

```
docs = loader.load()
```

```
# pip install docx2txt
```

YouTube Transcript

```
from langchain_community.document_loaders import YoutubeLoader
```

```
loader = YoutubeLoader.from_youtube_url("https://www.youtube.com/watch?v=...")
```

```
docs = loader.load()
```

```
# pip install youtube-transcript-api pytube
```

Notion

```
from langchain_community.document_loaders import NotionDBLoader
```

```
loader = NotionDBLoader(integration_token="...", database_id="...")
```



```
# Wikipedia
```

```
from langchain_community.document_loaders import WikipediaLoader
```

```
loader = WikipediaLoader(query="LangChain framework", load_max_docs=2)
```

```
docs = loader.load()
```

Document Object Anatomy

```
python
```

```
doc = docs[0]
```

```
print(doc.page_content) # "LangChain is a framework..."
```

```
print(doc.metadata)    # {'source': 'report.pdf', 'page': 0}
```

9. Text Splitters

Why Split?

LLMs have a context window limit (e.g., 128K tokens for GPT-4o). Large documents must be split into smaller chunks before being stored or sent to the LLM.

Key trade-off:

- Chunks too large → exceed context limit, high cost
- Chunks too small → lose context, poor answers

Recursive Character Text Splitter (Most Common)

```
python
```

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,    # Target chunk size in characters
    chunk_overlap=200,  # Overlap between chunks (preserves context)
    length_function=len, # How to measure length
    add_start_index=True, # Track position in original document
)

# Split documents
chunks = splitter.split_documents(docs)

# Split plain text
text = "Long article text here..."
chunks = splitter.split_text(text)

print(f'Number of chunks: {len(chunks)}')
print(f'First chunk: {chunks[0].page_content[:100]}')
```

How RecursiveCharacterTextSplitter Works

It tries to split on these separators in order:

1. `\n\n` (paragraphs) — preferred
2. `\n` (lines)
3. (words)
4. ```` (characters) — last resort

This preserves natural text boundaries.

TokenTextSplitter (Split by Tokens, Not Characters)

```
python
```

```
from langchain_text_splitters import TokenTextSplitter

# Split by actual token count (more accurate for LLM limits)
splitter = TokenTextSplitter(
    chunk_size=500, # tokens
    chunk_overlap=50,
)
chunks = splitter.split_documents(docs)

# pip install tiktoken
```

MarkdownHeaderTextSplitter (Structure-Aware)

```
python

from langchain_text_splitters import MarkdownHeaderTextSplitter

headers_to_split_on = [
    ("#", "Header 1"),
    ("##", "Header 2"),
    ("###", "Header 3"),
]

splitter = MarkdownHeaderTextSplitter(headers_to_split_on=headers_to_split_on)
splits = splitter.split_text(markdown_document)

# Each chunk has metadata: {'Header 1': 'Introduction', 'Header 2': 'Overview'}
```

Chunk Size Guidelines

Use Case	chunk_size	chunk_overlap
Short Q&A	256–512	50–100
General RAG	512–1024	100–200
Long-form analysis	1500–2000	200–400
Code files	500–1000	100–200

10. Embeddings & Vector Stores

What are Embeddings?

Embeddings convert text into numbers (vectors) that capture semantic meaning. Similar texts → similar vectors.

```
"I love dogs" → [0.2, 0.8, 0.1, 0.5, ...] (384 or 1536 numbers)
"I adore pets" → [0.21, 0.78, 0.12, 0.48, ...] (very similar numbers!)
"Stock market" → [0.9, 0.1, 0.7, 0.2, ...] (very different numbers)
```

Creating Embeddings

```
python

# OpenAI Embeddings (best quality, paid)
from langchain_openai import OpenAIEmbeddings
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

# HuggingFace (free, local)
from langchain_community.embeddings import HuggingFaceEmbeddings
embeddings = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")
# pip install sentence-transformers

# Google
from langchain_google_genai import GoogleGenerativeAIEmbeddings
embeddings = GoogleGenerativeAIEmbeddings(model="models/embedding-001")

# Test it
vector = embeddings.embed_query("What is machine learning?")
print(f"Vector dimensions: {len(vector)}") # 1536 for OpenAI small
```

Vector Stores

A vector store is a database optimized for storing and searching embeddings by similarity.

```
python
```

```
# FAISS (local, fast, good for dev/testing)
from langchain_community.vectorstores import FAISS

# Create from documents
vectorstore = FAISS.from_documents(chunks, embeddings)

# Save and load
vectorstore.save_local("faiss_index")
vectorstore = FAISS.load_local("faiss_index", embeddings, allow_dangerous_deserialization=True)

# Search
results = vectorstore.similarity_search("What is LangChain?", k=3)
results_with_scores = vectorstore.similarity_search_with_score("What is LangChain?", k=3)
```

```
python

# Chroma (local persistent DB, great for prototyping)
from langchain_chroma import Chroma
# pip install langchain-chroma chromadb

vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory="./chroma_db", # Persists to disk
    collection_name="my_docs"
)

# Load existing
vectorstore = Chroma(
    persist_directory="./chroma_db",
    embedding_function=embeddings,
    collection_name="my_docs"
)
```

```
python
```

```
# Pinecone (production-grade, cloud)
from langchain_pinecone import PineconeVectorStore
import pinecone

# pip install langchain-pinecone pinecone-client

pc = pinecone.Pinecone(api_key="your-api-key")
index = pc.Index("your-index-name")

vectorstore = PineconeVectorStore(index=index, embedding=embeddings)
vectorstore.add_documents(chunks)
```

Adding, Deleting, Updating Documents

```
python

# Add documents
vectorstore.add_documents(new_chunks)

# Add with custom IDs
ids = vectorstore.add_documents(chunks, ids=["doc1", "doc2", "doc3"])

# Delete by ID
vectorstore.delete(ids=["doc1", "doc2"])

# Get all stored documents
all_docs = vectorstore.get()
```

11. Retrievers & RAG

What is RAG?

Retrieval-Augmented Generation — instead of asking the LLM to answer from memory, you:

1. **Retrieve** relevant documents from your knowledge base
2. **Augment** the prompt with those documents
3. **Generate** an answer grounded in your data

User Question → Search Vector DB → Get Top K Relevant Chunks



LLM: "Answer this question using these documents: {chunks}"



Grounded, Accurate Answer

Basic Retriever

```
python
```

```
# Any vector store can be a retriever
```

```
retriever = vectorstore.as_retriever(  
    search_type="similarity",      # or "mmr" or "similarity_score_threshold"  
    search_kwargs={"k": 4}        # return top 4 docs  
)
```

```
# Retrieve
```

```
docs = retriever.invoke("What is LangChain?")
```

Search Types

```
python
```

```
# Similarity — returns top K most similar
```

```
retriever = vectorstore.as_retriever(search_type="similarity", search_kwargs={"k": 4})
```

```
# MMR (Maximum Marginal Relevance) — balances relevance AND diversity (avoids repetition)
```

```
retriever = vectorstore.as_retriever(search_type="mmr", search_kwargs={"k": 4, "fetch_k": 20})
```

```
# Threshold — only return docs above a similarity score
```

```
retriever = vectorstore.as_retriever(  
    search_type="similarity_score_threshold",  
    search_kwargs={"score_threshold": 0.7}  
)
```

Full RAG Chain

```
python
```

```

from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_core.runnables import RunnablePassthrough

# 1. Setup
llm = ChatOpenAI(model="gpt-4o-mini")
embeddings = OpenAIEmbeddings()
vectorstore = FAISS.load_local("faiss_index", embeddings, allow_dangerous_deserialization=True)
retriever = vectorstore.as_retriever(search_kwargs={"k": 4})

# 2. RAG Prompt
rag_prompt = ChatPromptTemplate.from_template("""
You are an assistant for question-answering tasks.
Use the following pieces of retrieved context to answer the question.
If you don't know the answer, say "I don't have enough information to answer this."
Always cite which document you used.

Context: {context}

Question: {question}

Answer:
""")

# 3. Format docs helper
def format_docs(docs):
    return "\n\n".join([f"[Source: {d.metadata.get('source', 'unknown')}] \n {d.page_content}"
                        for d in docs])

# 4. The RAG Chain
rag_chain = (
    {
        "context": retriever | format_docs,
        "question": RunnablePassthrough()
    }
    | rag_prompt
    | llm
    | StrOutputParser()
)

# 5. Ask a question

```



```
answer = rag_chain.invoke("What does LangChain help you build?")
print(answer)
```

Advanced: MultiQueryRetriever

```
python

# Generates multiple versions of your question to improve recall
from langchain.retrievers import MultiQueryRetriever

retriever = MultiQueryRetriever.from_llm(
    retriever=vectorstore.as_retriever(),
    llm=llm,
)

# Automatically generates:
# "What is LangChain?" → 3-5 different phrasings → merges results
```

Advanced: ContextualCompressionRetriever

```
python

# Retrieves docs, then compresses them to only include relevant parts
from langchain.retrievers import ContextualCompressionRetriever
from langchain.retrievers.document_compressors import LLMChainExtractor

compressor = LLMChainExtractor.from_llm(llm)
compression_retriever = ContextualCompressionRetriever(
    base_compressor=compressor,
    base_retriever=retriever
)

# Returns only the relevant sentences, not full chunks
```

Advanced: Ensemble Retriever (Hybrid Search)

```
python
```

```
# Combines keyword search (BM25) + vector search for best of both worlds
```

```
from langchain.retrievers import EnsembleRetriever
```

```
from langchain_community.retrievers import BM25Retriever
```

```
# pip install rank-bm25
```

```
bm25_retriever = BM25Retriever.from_documents(chunks)
```

```
bm25_retriever.k = 4
```

```
faiss_retriever = vectorstore.as_retriever(search_kwargs={"k": 4})
```

```
ensemble_retriever = EnsembleRetriever(  
    retrievers=[bm25_retriever, faiss_retriever],  
    weights=[0.4, 0.6]  
)
```

12. Memory

Why Memory?

By default, LLMs have no memory — every call is stateless. Memory modules let you:

- Remember what was said earlier in a conversation
- Personalize responses
- Build chatbots that feel natural

Conversation Buffer Memory (Store Everything)

```
python
```

```
from langchain.memory import ConversationBufferMemory

memory = ConversationBufferMemory(
    return_messages=True,    # Return as Message objects (for ChatModels)
    memory_key="chat_history", # Key name for MessagesPlaceholder
)

# Manually add to memory
memory.save_context(
    {"input": "My name is John"},
    {"output": "Nice to meet you, John!"}
)

# Get history
history = memory.load_memory_variables({})
print(history)
# {'chat_history': [HumanMessage("My name is John"), AIMessage("Nice to meet you, John!")]}
```

Building a Chatbot with Memory (LCEL Style)

```
python
```

```

from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
from langchain_core.messages import HumanMessage, AIMessage

llm = ChatOpenAI(model="gpt-4o-mini")

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant. Answer concisely."),
    MessagesPlaceholder(variable_name="chat_history"),
    ("human", "{input}"),
])

chain = prompt | llm

# Manual memory management (recommended in modern LangChain)
chat_history = []

def chat(user_input):
    response = chain.invoke({
        "input": user_input,
        "chat_history": chat_history
    })
    # Update history
    chat_history.append(HumanMessage(content=user_input))
    chat_history.append(AIMessage(content=response.content))
    return response.content

print(chat("My name is Alice")) # "Hello Alice! How can I help you?"
print(chat("What's my name?")) # "Your name is Alice."

```

RunnableWithMessageHistory (Production Pattern)

```
python
```

```

from langchain_core.runnables.history import RunnableWithMessageHistory
from langchain_community.chat_message_histories import ChatMessageHistory
from langchain_community.chat_message_histories import RedisChatMessageHistory

# In-memory store (for development)
store = {}

def get_session_history(session_id: str):
    if session_id not in store:
        store[session_id] = ChatMessageHistory()
    return store[session_id]

# Wrap your chain
chain_with_history = RunnableWithMessageHistory(
    chain,
    get_session_history,
    input_messages_key="input",
    history_messages_key="chat_history",
)

# Each session_id = separate conversation
config = {"configurable": {"session_id": "user_alice_123"}}
result1 = chain_with_history.invoke({"input": "Hi, I'm Alice"}, config=config)
result2 = chain_with_history.invoke({"input": "What's my name?"}, config=config)

# Redis for production (persistent across server restarts)
def get_redis_session_history(session_id: str):
    return RedisChatMessageHistory(session_id, url="redis://localhost:6379")

```

Memory Window (Limit Context Size)

```
python
```

```
from langchain_core.messages import trim_messages

# Keep only last 10 messages (5 turns)
trimmer = trim_messages(
    max_tokens=2000,
    strategy="last",
    token_counter=llm,
    include_system=True,
)

chain = prompt | trimmer | llm
```

Summary Memory (Compress Long Histories)

```
python

from langchain.memory import ConversationSummaryBufferMemory

# Keeps recent messages + a running summary of older ones
memory = ConversationSummaryBufferMemory(
    llm=llm,
    max_token_limit=500, # When history exceeds this, summarize old messages
    return_messages=True
)
```

13. Agents & Tools

What is an Agent?

An agent uses an LLM to **decide what to do next**. Instead of following a fixed chain, it:

1. Looks at the task
2. Decides which tool to use
3. Uses the tool
4. Looks at the result
5. Decides what to do next (loop until done)

What is a Tool?

A tool is a function the agent can call. Examples:

- Search the web
- Run Python code
- Query a database
- Call an API
- Calculate something

Built-in Tools

```
python
```

```
from langchain_community.tools import WikipediaQueryRun, DuckDuckGoSearchRun
from langchain_community.utilities import WikipediaAPIWrapper, DuckDuckGoSearchAPIWrapper
```

```
# Wikipedia
```

```
wiki = WikipediaQueryRun(api_wrapper=WikipediaAPIWrapper())
result = wiki.invoke("LangChain framework")
```

```
# Web Search
```

```
search = DuckDuckGoSearchRun()
result = search.invoke("latest AI news 2025")
```

```
# Python REPL (run Python code!)
```

```
from langchain_experimental.tools import PythonREPLTool
python_repl = PythonREPLTool()
result = python_repl.invoke("print(2 ** 10)") # "1024"
```

Creating Custom Tools

```
python
```

```

from langchain_core.tools import tool
from pydantic import BaseModel

# Simple decorator approach
@tool
def get_weather(city: str) -> str:
    """Get the current weather for a city. Input should be a city name."""
    # Your actual weather API call here
    return f"The weather in {city} is sunny, 25°C"

# With input validation
class WeatherInput(BaseModel):
    city: str
    unit: str = "celsius"

@tool(args_schema=WeatherInput)
def get_detailed_weather(city: str, unit: str = "celsius") -> str:
    """Get detailed weather. Always specify the city and unit."""
    return f"Weather in {city}: 25° {unit}, humidity 60%"

# Test the tool directly
print(get_weather.invoke("London"))
print(get_weather.name)      # "get_weather"
print(get_weather.description) # The docstring

```

Building an Agent

```
python
```



```
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool
from langchain.agents import create_tool_calling_agent, AgentExecutor
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder
```

```
# 1. Tools
```

```
@tool
```

```
def multiply(a: int, b: int) -> int:
    """Multiply two numbers together."""
    return a * b
```

```
@tool
```

```
def add(a: int, b: int) -> int:
    """Add two numbers together."""
    return a + b
```

```
tools = [multiply, add]
```

```
# 2. LLM
```

```
llm = ChatOpenAI(model="gpt-4o", temperature=0)
```

```
# 3. Prompt
```

```
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful math assistant."),
    ("human", "{input}"),
    MessagesPlaceholder(variable_name="agent_scratchpad"), # Agent's thinking space
])
```

```
# 4. Create Agent
```

```
agent = create_tool_calling_agent(llm, tools, prompt)
```

```
# 5. Executor (runs the agent loop)
```

```
agent_executor = AgentExecutor(
    agent=agent,
    tools=tools,
    verbose=True, # See the agent's reasoning
    max_iterations=10,
    handle_parsing_errors=True
)
```

```
# 6. Run it
```

```
result = agent_executor.invoke({
    "input": "What is (3 + 5) * 12?"
})
```

```
})  
print(result["output"])  
# Agent: I'll first add 3+5=8, then multiply 8*12=96  
# Answer: 96
```

Agent Reasoning (Verbose Output)

```
Entering AgentExecutor chain...  
Thought: I need to add 3+5 first  
Action: add  
Action Input: {"a": 3, "b": 5}  
Observation: 8  
Thought: Now I multiply 8 by 12  
Action: multiply  
Action Input: {"a": 8, "b": 12}  
Observation: 96  
Thought: I have the final answer  
Final Answer: The result is 96
```

ReAct Agent Pattern

ReAct = Reasoning + Acting The agent thinks step by step, alternating between thinking and taking actions. This is the most common agent pattern.

Agent with Memory

```
python
```

```

from langchain_core.messages import HumanMessage, AIMessage

agent_executor_with_memory = AgentExecutor(
    agent=agent,
    tools=tools,
    verbose=True
)

# Pass chat history manually
chat_history = []
result = agent_executor_with_memory.invoke({
    "input": "My favorite number is 42",
    "chat_history": chat_history
})
chat_history.extend([
    HumanMessage(content="My favorite number is 42"),
    AIMessage(content=result["output"])
])

result = agent_executor_with_memory.invoke({
    "input": "What is my favorite number doubled?",
    "chat_history": chat_history
})
print(result["output"]) # "Your favorite number is 42, so doubled it's 84"

```

14. LangChain Expression Language (LCEL)

What is LCEL?

LCEL is the modern, recommended way to build LangChain pipelines using the `|` (pipe) operator. It's inspired by Unix pipes.

Why LCEL?

- Cleaner, more readable code
- Built-in async, streaming, batch support
- Automatic parallel execution where possible
- Better tracing with LangSmith
- Composable and testable

LCEL Cheat Sheet

```
python

from langchain_core.runnables import (
    RunnablePassthrough, # Pass input unchanged
    RunnableParallel,    # Run in parallel
    RunnableLambda,      # Custom function
    RunnableBranch,      # Conditional routing
)

# Basic chain
chain = prompt | llm | parser

# Pass-through (preserve original input)
chain = RunnableParallel(
    answer=prompt | llm | parser,
    original=RunnablePassthrough()
)

# Lambda (custom transformation)
chain = RunnableLambda(lambda x: x.upper()) | prompt | llm

# Branch (conditional logic)
chain = RunnableBranch(
    (lambda x: "technical" in x["topic"], technical_prompt | llm),
    (lambda x: "creative" in x["topic"], creative_prompt | llm),
    default_prompt | llm # Default case
)
```

Inspecting Your Chain

```
python

# See the structure
chain.get_graph().print_ascii()

# Get input/output schemas
print(chain.input_schema.schema())
print(chain.output_schema.schema())
```

Streaming

```
python
```

```
# Stream tokens
for token in chain.stream({"question": "Explain quantum computing"}):
    print(token, end="", flush=True)

# Async streaming (for FastAPI/web servers)
async def stream_response():
    async for token in chain.astream({"question": "Hello"}):
        yield token
```

15. LangSmith

What is LangSmith?

LangSmith is Anthropic's — no, **Langchain's** — observability platform for LLM apps. It lets you:

- **Trace** every call (see what prompt was sent, what came back, how long it took, how much it cost)
- **Debug** failures and unexpected outputs
- **Evaluate** your app's quality with test datasets
- **Monitor** production performance

Setup

```
python

# Set these in your .env
LANGCHAIN_TRACING_V2=true
LANGCHAIN_API_KEY=ls__your_key
LANGCHAIN_PROJECT=my-rag-app
```

Once enabled, **every LangChain call is automatically traced** — no code changes needed!

Manual Tracing

```
python

from langsmith import traceable

@traceable(name="My Custom Function")
def process_query(query: str) -> str:
    result = rag_chain.invoke(query)
    return result
```

Evaluation

```
python

from langsmith.evaluation import evaluate, LangChainStringEvaluator

# Create test dataset in LangSmith UI, then evaluate
results = evaluate(
    lambda inputs: rag_chain.invoke(inputs["question"]),
    data="my-test-dataset",
    evaluators=[
        LangChainStringEvaluator("correctness"),
        LangChainStringEvaluator("helpfulness"),
    ]
)
```

16. LangServe

What is LangServe?

LangServe turns any LangChain Runnable into a REST API with a single function call.

It auto-generates:

- `POST /chain/invoke` — single request
- `POST /chain/stream` — streaming response
- `POST /chain/batch` — batch requests
- `GET /chain/playground` — interactive UI to test your chain

Installation

```
bash

pip install langserve[all] # or: pip install "langserve[server]"
```

Creating an API

```
python
```

```
# serve.py
from fastapi import FastAPI
from langserve import add_routes
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

app = FastAPI(title="My AI API", version="1.0")

# Define your chain
chain = (
    ChatPromptTemplate.from_template("Tell me a joke about {topic}")
    | ChatOpenAI(model="gpt-4o-mini")
    | StrOutputParser()
)

# Add routes (this is all you need!)
add_routes(app, chain, path="/joke")

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

```
bash

# Run it
python serve.py

# Test it
curl -X POST http://localhost:8000/joke/invoke \
  -H "Content-Type: application/json" \
  -d '{"input": {"topic": "Python"}}'

# Or visit: http://localhost:8000/joke/playground
```

Client

```
python
```

```
from langserve import RemoteRunnable

# Use your deployed chain like a local chain
remote_chain = RemoteRunnable("http://localhost:8000/joke/")
result = remote_chain.invoke({"topic": "AI"})
print(result)
```

17. LangGraph

What is LangGraph?

LangGraph is LangChain's framework for building **stateful, multi-agent workflows** as directed graphs. It's used for:

- Complex agents that need to loop and backtrack
- Multi-agent systems (e.g., researcher + writer + reviewer)
- Human-in-the-loop workflows
- Long-running automated pipelines

Core Concepts

State → Shared data structure all nodes read/write
Node → A function that reads state and returns updates
Edge → Connection between nodes (can be conditional)
Graph → The workflow definition

Simple LangGraph Example

```
python
```



```

from langgraph.graph import StateGraph, START, END
from langchain_openai import ChatOpenAI
from typing import TypedDict, Annotated
import operator

# 1. Define State
class AgentState(TypedDict):
    question: str
    research: str
    answer: str

# 2. Define Nodes (functions)
llm = ChatOpenAI(model="gpt-4o-mini")

def research_node(state: AgentState) -> dict:
    """Research the question"""
    response = llm.invoke(f'Research this topic briefly: {state["question"]}')
    return {"research": response.content}

def answer_node(state: AgentState) -> dict:
    """Answer using research"""
    response = llm.invoke(
        f'Question: {state["question"]}\nResearch: {state["research"]}\nAnswer:'
    )
    return {"answer": response.content}

# 3. Build Graph
builder = StateGraph(AgentState)

builder.add_node("research", research_node)
builder.add_node("answer", answer_node)

builder.add_edge(START, "research")
builder.add_edge("research", "answer")
builder.add_edge("answer", END)

graph = builder.compile()

# 4. Run it
result = graph.invoke({"question": "What is quantum entanglement?"})
print(result["answer"])

```

Conditional Edges (Decision Making)

```
python

def route_question(state: AgentState) -> str:
    """Decide which path to take"""
    if "math" in state["question"].lower():
        return "math_node"
    else:
        return "general_node"

builder.add_conditional_edges(
    "classifier",
    route_question,
    {
        "math_node": "math_node",
        "general_node": "general_node"
    }
)
```

ReAct Agent in LangGraph

```
python

from langgraph.prebuilt import create_react_agent

agent = create_react_agent(
    model=llm,
    tools=[search_tool, calculator_tool],
)

result = agent.invoke({"messages": [("human", "What's 15% of 280?")]})
```

LangGraph vs. AgentExecutor

Feature	AgentExecutor	LangGraph
Use case	Simple agents	Complex, stateful agents
Control flow	Linear loop	Custom graph
Multi-agent	No	Yes
Human-in-loop	Limited	First-class support
Streaming	Partial	Full

Feature	AgentExecutor	LangGraph
Recommended for	Quick prototypes	Production systems

18. Production & Enterprise Patterns

1. Rate Limiting & Cost Control

```
python

# Limit requests per minute
from langchain_core.rate_limiters import InMemoryRateLimiter

rate_limiter = InMemoryRateLimiter(
    requests_per_second=10,
    check_every_n_seconds=0.1,
    max_bucket_size=10,
)

llm = ChatOpenAI(model="gpt-4o", rate_limiter=rate_limiter)
```

2. Caching (Save Money & Speed Up Responses)

```
python

from langchain_core.globals import set_llm_cache
from langchain_community.cache import InMemoryCache, SQLiteCache, RedisCache

# In-memory (dev/testing)
set_llm_cache(InMemoryCache())

# SQLite (single server, persistent)
set_llm_cache(SQLiteCache(database_path=".langchain.db"))

# Redis (multi-server, production)
from redis import Redis
set_llm_cache(RedisCache(redis_=Redis.from_url("redis://localhost:6379"))))

# Now identical prompts return cached responses instantly (no API call, no cost)
```

3. Callbacks & Monitoring

```
python

from langchain_core.callbacks import BaseCallbackHandler

class CostTracker(BaseCallbackHandler):
    def __init__(self):
        self.total_tokens = 0
        self.total_cost = 0.0

    def on_llm_end(self, response, **kwargs):
        usage = response.llm_output.get("token_usage", {})
        input_tokens = usage.get("prompt_tokens", 0)
        output_tokens = usage.get("completion_tokens", 0)
        self.total_tokens += input_tokens + output_tokens
        # GPT-4o-mini pricing
        self.total_cost += (input_tokens * 0.00015 + output_tokens * 0.0006) / 1000

tracker = CostTracker()
llm = ChatOpenAI(model="gpt-4o-mini", callbacks=[tracker])

result = llm.invoke("Tell me about AI")
print(f"Cost: ${tracker.total_cost:.6f}")
print(f"Tokens used: {tracker.total_tokens}")
```

4. Async for High Throughput

```
python

import asyncio
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o-mini")

async def process_batch(questions: list[str]):
    tasks = [llm.ainvoke(q) for q in questions]
    responses = await asyncio.gather(*tasks)
    return [r.content for r in responses]

# Process 100 questions concurrently
questions = ["Q1", "Q2", ..., "Q100"]
answers = asyncio.run(process_batch(questions))
```

5. Secrets Management

```
python

# Never hardcode API keys!
# Use environment variables + a secrets manager in production

# AWS Secrets Manager
import boto3
import json

def get_secret(secret_name: str) -> dict:
    client = boto3.client("secretsmanager", region_name="us-east-1")
    response = client.get_secret_value(SecretId=secret_name)
    return json.loads(response["SecretString"])

secrets = get_secret("my-app/openai")
llm = ChatOpenAI(api_key=secrets["OPENAI_API_KEY"])
```

6. Multi-tenancy Pattern

```
python

# Different customers → different vector stores/configs
class MultiTenantRAG:
    def __init__(self):
        self.llm = ChatOpenAI(model="gpt-4o-mini")
        self.embedding = OpenAIEmbeddings()

    def get_retriever(self, tenant_id: str):
        # Each tenant has their own Chroma collection
        vectorstore = Chroma(
            collection_name=f"tenant_{tenant_id}",
            embedding_function=self.embedding,
            persist_directory=f"./data/{tenant_id}"
        )
        return vectorstore.as_retriever()

    def query(self, tenant_id: str, question: str) -> str:
        retriever = self.get_retriever(tenant_id)
        rag_chain = retriever | format_docs | rag_prompt | self.llm | StrOutputParser()
        return rag_chain.invoke(question)
```

7. Guardrails & Safety

```
python

from langchain_core.runnables import RunnableLambda

BANNED_TOPICS = ["competitor", "confidential", "lawsuit"]

def check_input(input_dict: dict) -> dict:
    question = input_dict["question"].lower()
    if any(topic in question for topic in BANNED_TOPICS):
        raise ValueError(f"Question contains restricted topic")
    return input_dict

def check_output(output: str) -> str:
    if "competitor_name" in output.lower():
        return "I can't discuss competitors. How else can I help?"
    return output

safe_chain = (
    RunnableLambda(check_input)
    | rag_chain
    | RunnableLambda(check_output)
)
```

8. RAG Evaluation Metrics

```
python
```

```
# Key metrics to track in production RAG:
```

```
"""
```

1. Context Recall — Did we retrieve all the necessary information?
2. Context Precision — Was retrieved info actually relevant? (no noise)
3. Answer Relevancy — Does the answer address the question?
4. Faithfulness — Is the answer grounded in retrieved context? (no hallucination)

```
"""
```

```
# Use RAGAS library for automated RAG evaluation
```

```
# pip install ragas
```

```
from ragas import evaluate
```

```
from ragas.metrics import faithfulness, answer_relevancy, context_precision
```

```
results = evaluate(  
    dataset=test_dataset,  
    metrics=[faithfulness, answer_relevancy, context_precision],  
)
```

9. Document Ingestion Pipeline (Production)

```
python
```

```

# Full production ingestion pipeline
import hashlib
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_community.document_loaders import PyPDFLoader

class DocumentIngestionPipeline:
    def __init__(self, vectorstore, embeddings):
        self.vectorstore = vectorstore
        self.splitter = RecursiveCharacterTextSplitter(
            chunk_size=1000,
            chunk_overlap=200,
        )
        # Track ingested documents to avoid duplicates
        self.ingested = set()

    def ingest(self, file_path: str):
        # 1. Check if already ingested (deduplication)
        file_hash = hashlib.md5(open(file_path, 'rb').read()).hexdigest()
        if file_hash in self.ingested:
            print(f'Skipping {file_path} — already ingested')
            return

        # 2. Load
        loader = PyPDFLoader(file_path)
        docs = loader.load()

        # 3. Split
        chunks = self.splitter.split_documents(docs)

        # 4. Add metadata
        for chunk in chunks:
            chunk.metadata["file_hash"] = file_hash
            chunk.metadata["ingested_at"] = "2025-01-01"

        # 5. Store
        self.vectorstore.add_documents(chunks)
        self.ingested.add(file_hash)
        print(f'Ingested {len(chunks)} chunks from {file_path}')

```

10. FastAPI + LangChain Production App

python


```

# main.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from contextlib import asynccontextmanager
import asyncio

app = FastAPI()
rag_chain = None

@asynccontextmanager
async def lifespan(app: FastAPI):
    # Initialize on startup
    global rag_chain
    rag_chain = build_rag_chain() # Your chain setup
    yield
    # Cleanup on shutdown

app = FastAPI(lifespan=lifespan)

class QueryRequest(BaseModel):
    question: str
    session_id: str = "default"

class QueryResponse(BaseModel):
    answer: str
    sources: list[str]

@app.post("/query", response_model=QueryResponse)
async def query(request: QueryRequest):
    try:
        result = await rag_chain.ainvoke({
            "question": request.question,
            "session_id": request.session_id
        })
        return QueryResponse(
            answer=result["answer"],
            sources=result["sources"]
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@app.get("/health")

```

```
def health():  
    return {"status": "healthy"}
```

19. Interview Q&A

Beginner Level

Q: What is LangChain and why would you use it? A: LangChain is an open-source framework for building LLM-powered applications. You'd use it instead of raw API calls because it provides: ready-made abstractions for prompts, chains, and agents; integrations with 100+ tools and databases; and composability through LCEL that makes complex pipelines readable and maintainable.

Q: What is the difference between an LLM and a Chat Model in LangChain? A: An LLM takes a string and returns a string (older style). A Chat Model takes a list of messages (SystemMessage, HumanMessage, AIMessage) and returns a message. Chat models are the modern standard and most APIs (GPT-4, Claude) are chat models under the hood.

Q: What is LCEL and why is it preferred? A: LCEL (LangChain Expression Language) is a declarative way to compose Runnables using the `|` pipe operator. It's preferred because it automatically supports streaming, async, batching, and parallel execution with no extra code. It also integrates natively with LangSmith for tracing.

Q: Explain what RAG is and why it's important. A: RAG (Retrieval-Augmented Generation) is a pattern where you retrieve relevant documents from a knowledge base and inject them into the LLM's prompt before generating an answer. It's important because: (1) LLMs have knowledge cutoff dates — RAG keeps them current. (2) It grounds answers in real data, reducing hallucinations. (3) It's cheaper than fine-tuning for domain-specific knowledge.

Intermediate Level

Q: What is the difference between chunk_size and chunk_overlap in text splitting? How do you choose them? A: `chunk_size` is the maximum number of characters per chunk. `chunk_overlap` is how many characters are shared between consecutive chunks. Overlap prevents answers from being split across chunks. Choose `chunk_size` based on your LLM's context window and the granularity of information — smaller for precise Q&A (256-512), larger for summaries (1500-2000). Overlap should be ~20% of `chunk_size`.

Q: What is the difference between similarity search and MMR? A: Similarity search returns the K most similar documents to the query, which can result in redundant/repetitive results. MMR (Maximum Marginal Relevance) balances relevance with diversity — it avoids returning near-duplicate chunks by penalizing documents too similar to already-selected ones. Use MMR when you want diverse coverage of a topic.

Q: How does an Agent decide which tool to use? A: Modern agents use "tool calling" (function calling). The LLM is given descriptions of available tools and generates a structured tool invocation (name + arguments) as part of its output. The AgentExecutor intercepts this, calls the actual function, injects the result back as a "tool observation," and the LLM continues until it produces a final answer.

Q: What is the difference between RunnablePassthrough and RunnableLambda? A: `RunnablePassthrough` simply passes the input unchanged to the next step — useful for preserving the original input alongside transformed versions. `RunnableLambda` wraps an arbitrary Python function, making it a Runnable that can be used in a chain.

Production & Enterprise Level

Q: How would you handle LLM rate limits and timeouts in production? A: (1) Use `.with_retry()` with exponential backoff for transient failures. (2) Use `.with_fallbacks()` to fall back to a cheaper/different model. (3) Use `InMemoryRateLimiter` or a Redis-backed rate limiter to control request rates. (4) Set appropriate `timeout` and `max_retries` on the LLM. (5) Implement circuit breakers for catastrophic failures.

Q: How do you reduce LLM costs in production? A: (1) **Caching** — set up `RedisCache` so identical prompts never hit the API twice. (2) **Smaller models** — use GPT-4o-mini for simpler tasks, GPT-4o only when needed. (3) **Prompt optimization** — shorter system prompts, filtered context. (4) **Output limits** — set appropriate `max_tokens`. (5) **Batch processing** — use `.batch()` for bulk tasks. (6) **Semantic caching** — cache similar (not just identical) queries.

Q: How do you evaluate a RAG pipeline in production? A: Use the RAGAS framework to measure: (1) **Faithfulness** — is the answer grounded in retrieved docs? (2) **Context Precision** — are retrieved chunks relevant? (3) **Context Recall** — did we retrieve all necessary information? (4) **Answer Relevancy** — does the answer actually address the question? Also monitor: retrieval latency, LLM latency, end-to-end latency, cost per query, and user feedback (thumbs up/down).

Q: How would you build a multi-tenant RAG system? A: (1) Give each tenant their own vector store collection (Chroma collection name or Pinecone namespace). (2) Add `tenant_id` as metadata on all documents for filtering. (3) Use metadata filtering in retrieval: `retriever.search_kkwargs={"filter": {"tenant_id": "ABC"}}`. (4) Store API keys and configurations per tenant. (5) Use LangSmith projects per tenant for isolated monitoring.

Q: What are the limitations of LangChain and when would you NOT use it? A: (1) **Abstraction overhead** — for simple single-call apps, direct API calls are lighter. (2) **Rapid changes** — LangChain evolves fast; pinning versions is critical in production. (3) **Learning curve** — LCEL and the ecosystem can be complex for new developers. (4) **Debugging** — multi-step chains can be hard to debug without LangSmith. You'd skip LangChain for: simple one-off scripts, when you need maximum control over every API call, or when using frameworks like LlamaIndex that better suit your specific use case.

Q: How do you handle long documents that don't fit in context? A: (1) **RAG** — split and retrieve only relevant chunks. (2) **Map-Reduce** — process each chunk separately, then combine (use `load_summarize_chain` with `chain_type="map_reduce"`). (3) **Refine** — process sequentially, refining the answer with each chunk. (4)

Hierarchical summarization — summarize chunks, then summarize summaries. (5) **Large context models** — use models like Gemini 1.5 Pro (1M tokens) for very long documents.

Q: How does LangGraph differ from AgentExecutor and when should you use it? A: AgentExecutor runs a simple loop: think → act → observe → repeat. It's good for straightforward tasks. LangGraph is a stateful graph where nodes are functions and edges define control flow. Use LangGraph when you need: conditional branching, cycles (loops with state), multiple cooperating agents, human-in-the-loop interrupts, or long-running workflows with checkpointing/resuming.

Q: Explain semantic caching and how to implement it. A: Unlike exact-match caching, semantic caching stores LLM responses and uses embedding similarity to serve cached answers for semantically similar (but not identical) queries. For example, "What is ML?" and "Explain machine learning" would hit the same cache entry. Implementation with LangChain: use `GPTCache` or `LangChainCache` with a vector similarity check before hitting the LLM API.

20. Resources & Learning Path

Official Documentation

Resource	URL	Best For
LangChain Docs	docs.langchain.com	Primary reference
LangChain Python API	api.python.langchain.com	API details
LangSmith Docs	docs.smith.langchain.com	Observability
LangGraph Docs	langchain-ai.github.io/langgraph	Agents/graphs
LangChain GitHub	github.com/langchain-ai/langchain	Source & examples

Video Courses (Free)

Resource	URL	Best For
LangChain YT Channel	youtube.com/@LangChain	Official tutorials
Greg Kamradt's LangChain course	youtube.com/@DataIndependent	In-depth walkthroughs
Krish Naik's LangChain Playlist	YouTube search	Beginner-friendly
DeepLearning.AI Short Courses	learn.deeplearning.ai	Hands-on projects

DeepLearning.AI Courses (Highly Recommended)

1. **LangChain for LLM Application Development** (with Harrison Chase)
2. **LangChain: Chat with Your Data**
3. **Building and Evaluating Advanced RAG**
4. **Functions, Tools and Agents with LangChain**
5. **AI Agents in LangGraph**

GitHub Repositories to Study

langchain-ai/langchain	→ Main repo (study examples/)
langchain-ai/langgraph	→ LangGraph examples
langchain-ai/langchain-cookbook	→ Production patterns
gkamradt/langchain-tutorials	→ Great beginner notebooks
mistralai/cookbook	→ Multi-LLM patterns

Suggested Learning Path (4-Week Plan)

Week 1 — Foundations:

- Install LangChain, run your first chain
- Understand LLMs, Chat Models, Prompt Templates
- Build a simple Q&A bot with memory
- Read: Chapters 1-7 of this guide

Week 2 — RAG System:

- Build a PDF Q&A system end-to-end
- Experiment with different splitters, chunk sizes
- Try FAISS and Chroma vector stores
- Implement basic and advanced retrievers

Week 3 — Agents & Advanced:

- Build an agent with 3+ tools
- Implement a multi-step LangGraph workflow
- Add LangSmith tracing to all your projects
- Deploy a chain with LangServe

Week 4 — Production:

- Add caching, retry logic, fallbacks
 - Implement evaluation with RAGAS
 - Build a FastAPI app around your RAG system
 - Explore multi-agent patterns in LangGraph
-

Quick Reference Card

python

1. Basic Chain

```
chain = ChatPromptTemplate.from_template("{q}") | ChatOpenAI() | StrOutputParser()
chain.invoke({"q": "Hello"})
```

2. RAG in 5 lines

```
vectorstore = FAISS.from_documents(splits, OpenAIEmbeddings())
retriever = vectorstore.as_retriever()
rag_chain = {"context": retriever | format_docs, "question": RunnablePassthrough()} | prompt | llm | StrOutputParser()
```

3. Agent

```
agent_executor = AgentExecutor(agent=create_tool_calling_agent(llm, tools, prompt), tools=tools)
agent_executor.invoke({"input": "Search for latest AI news"})
```

4. Memory

```
chain_with_history = RunnableWithMessageHistory(chain, get_session_history, input_messages_key="input", history_message_key="history")
```

5. Structured Output

```
class MySchema(BaseModel): field: str
structured = llm.with_structured_output(MySchema)
```

6. Streaming

```
for chunk in chain.stream({"question": "Hello"}): print(chunk, end="")
```

7. Caching

```
set_llm_cache(RedisCache(redis_=Redis.from_url("redis://localhost:6379")))
```

8. Retry + Fallback

```
robust = llm.with_retry(stop_after_attempt=3).with_fallbacks([cheap_llm])
```

This guide covers LangChain as of 2025 (v0.3+). Always check the official docs at docs.langchain.com for the latest updates.